

CORS & Preflight Requests – Concept + Practical Investigation

Part 1 – Concept Understanding

1. What is CORS and why it exists

Cors:

Cross origin resource sharing manages the communication between two origins and makes sure they are not violating the privacy of one another.

Why it exists:

It exists to protect the end users from the hackers who are trying to use the user's active sessions and cookies and make requests to the server pretending to be the legitimate user without the user's permission. It protects the user by verifying the response headers of the server and rejecting the user request in case of a mismatch.

2. What is an Origin?

Origin represents the address of the web page with which we can uniquely identify it. The address consists of three parts(scheme, domain and port)

Same-Origin Example: <https://example.com/page1> and <https://example.com/page2> match perfectly across all three components.

Cross-Origin Example: <https://example.com> and <http://example.com> (different scheme) or <https://api.example.com> (different host sub-domain) are flagged as entirely different origins.

3. What is a Cross-Origin Request?

This is when one origin makes a request to a completely different origin. The browser intercepts this action and makes sure the destination origin trusts the source origin and ensures the security in the process through CORS.

4. Simple Requests vs Preflighted Requests

Simple Requests:

These are safe and less impactful requests such as GET. They do not modify the website or delete any records. So the browser allows these requests to go to the server without asking the server's permission.

Preflighted Requests

These are less safe and more impactful requests which modify or delete the records of the database or change the data. So the browser, before allowing these impactful actions, pre-checks with the website and confirms whether this action is safe to perform. We call these requests **Preflighted Requests**

5. What is a Preflight OPTIONS Request

This is the actual HTTP request type sent by the browser to the server in case of the preflight request. Through this options method, it sends the metadata information about the request to the server. The server responds with the metadata headers showing the acceptable actions and the browser after inspecting them, chooses to accept or reject the request from the user.

6. What Triggers a Preflight?

Preflight is triggered when using,

non-simple HTTP methods (**PUT, DELETE, PATCH**)
sending custom headers (**like Authorization or X-Auth-Token**)
specifying a non-simple content-type like **application/json**.

7. What are Key CORS Response Headers?

- Access-Control-Allow-Origin:

Tells the browser exactly which external website domains are legally allowed to access the server.

- Access-Control-Allow-Methods:

A comma-separated list specifying which HTTP operations (e.g., GET, POST, PUT) the server permits from outside origins

- Access-Control-Allow-Headers

Specifies the exact custom headers the frontend is allowed to attach to its incoming requests.

- Access-Control-Allow-Credentials (and rule against using ***** with credentials)

Tells the browser if it's safe to expose cookies/tokens; it strictly cannot be used with a wildcard (*) because the browser demands an explicitly named, trusted origin when handling sensitive user sessions.

8. Why it works in Postman or curl but not in the browser ?

Postman or curl do not have a browsing engine or a shared user session across different tabs. It is just a standalone api testing client. So the CORS which is only a strict browser only security feature is getting completely bypassed here and so this works fine for postman and in the browser, with all those CORS enabled, it may not work.

Part 2 – Talk to Senior Developer

Problem:

They were working on a project and in the backend and configured the response header of the server as **Access-Control-Allow-Origin : *** and received many errors due to this.

Also they were working on a site which had Content-Security-Policy: default-src 'self'. This makes the site accept requests coming from the same website with all the address components being the same and rejects other origins. They were trying to read this from another origin and they could not because of these rules.

Solution Taken:

Instead of keeping the **Access-Control-Allow-Origin : ***, they listed a specific list of valid origins and then it worked well for those listed origins.

Modify the header property and make it accept requests from other origins.

My Takeaway:

I will always list the valid set of origins which can access my site and would try not to use the wildcard.

When I am building a site on one origin that will be used on another origin, I will make sure I do not force it to allow only the request from the same origin and configure the CORS properly so that my other origins can access it.